HARMONY

Shared Living Management & Conflict Resolution Platform

Minor Project Final Presentation (BCA-508) | Full-Stack System for Automated Expense Splitting, Effort Gamification & Household Transparency

ACADEMIC CREDENTIALS

Course & Code: BCA 5th Semester | Minor Project (BCA-508)

Candidate Name: Lakshya Shukla

University Roll No: 240919106121 (Section B)

College Email: lakshyashukla.bcr24257@imsnoida.com

Project Guide: Mr. Nihal Kumar, Assistant Professor

Review Panel: Panel 2 (Room DG4) | Final Presentation Day

Institution: IMS Noida | Affiliated to CCS University, Meerut

MANDATORY FINAL PRESENTATION LIVE LINKS

Clickable Production Access (As Mandated by Department):

[LIVE GITHUB PAGES] lakshya212.github.io/HARMONY

[LIVE VERCEL DEPLOYMENT] harmony-household.vercel.app

[PUBLIC GITHUB REPOSITORY & CODE] github.com/lakshya212/HARMONY

Project Overview & Problem Statement

THE PROBLEM CONTEXT

Shared Household Friction:

• Asymmetric Financial Burden: Flatmates and multi-earner households frequently lose track of who paid for groceries, utilities, and internet bills, causing awkward disputes.

• The 'Chore Trap': Traditional chore tracking ignores effort disparity (e.g., taking out trash vs deep cleaning bathroom treated equally).

• Unreliable Memory: Informal verbal agreements lead to resentment and roommate conflicts.

THE HARMONY SOLUTION

Mathematical Transparency:

• Automated Split Engine: Instantly splits shared expenses equally or by shares, updating a real-time Net Balance Ledger.

• Gamified Effort Points: Chores carry calibrated effort points (10-30 pts). System highlights top contributors dynamically.

• Unified Dashboard: Gives roommates an instant high-level view of who owes what and who completed pending duties.

• Zero Setup Overhead: Instant household code generation for roommates.

ACADEMIC OBJECTIVES

BCA-508 Minor Project Scope:

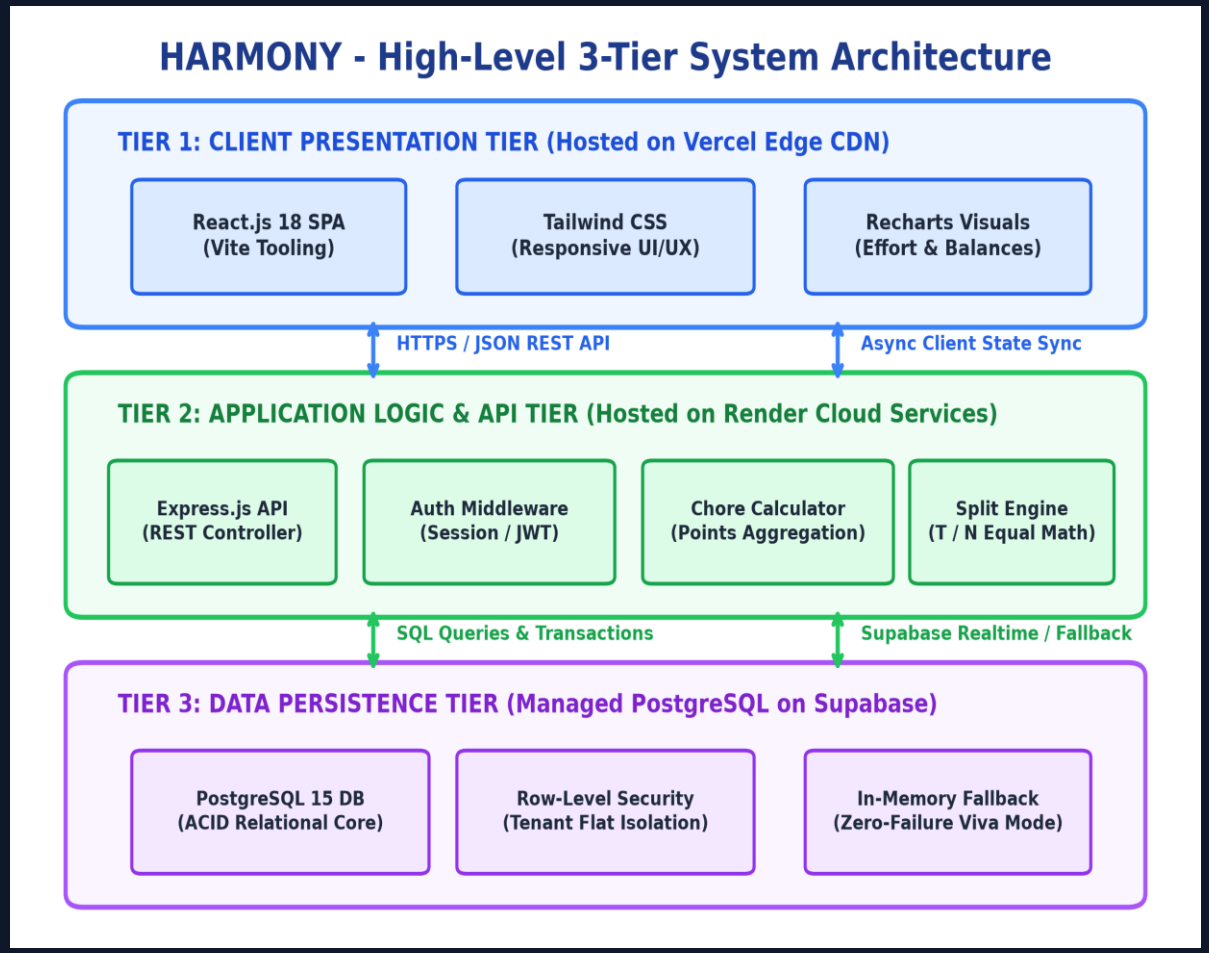
• Full-Stack Architecture: Modern React Single Page Application (SPA) consuming decoupled Node.js RESTful APIs.

• Relational Integrity: Enforcing 3NF relational schemas with Foreign Keys and transactional integrity in PostgreSQL.

• Production Cloud Deployment: Automated CD workflow with Vercel Edge hosting and cloud backend services.

• Defensive Reliability: Dual-mode data resilience preventing presentation failure during network outages.

Three-Tier System Architecture & Tech Stack



TECHNICAL IMPLEMENTATION

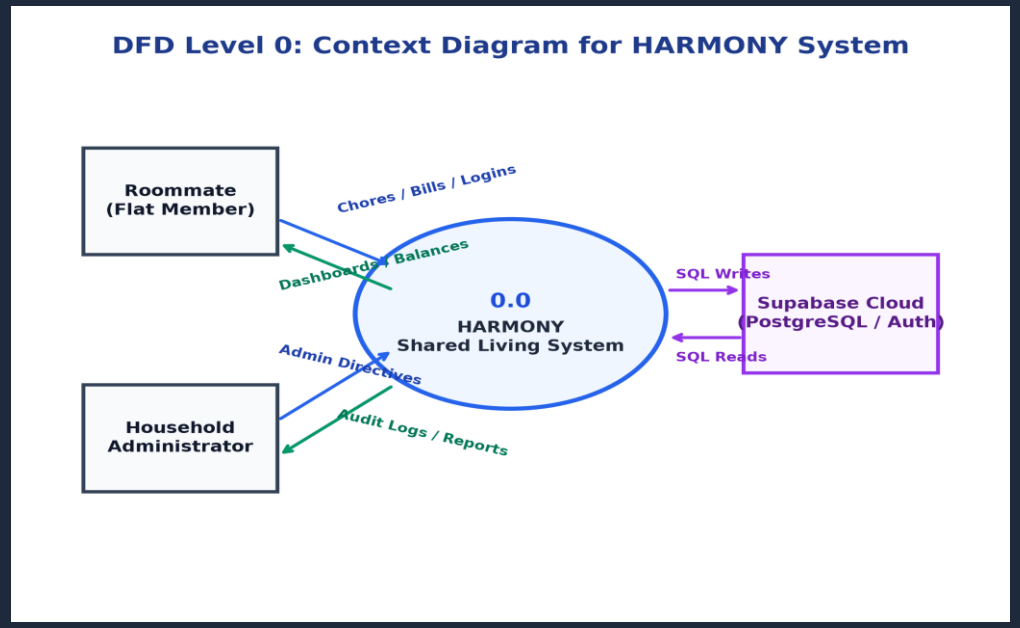
Presentation Tier (Frontend):
- React 18 with Vite build tooling for sub-second HMR.
- Tailwind CSS for modern responsive dark-mode styling.
- Lucide React icon library and client-side SPA routing.

Application Tier (Backend API):
- Node.js & Express.js RESTful API architecture.
- Middleware: CORS protection, JSON body parsing, and request logging.
- Strict route isolation: /api/auth, /api/chores, /api/expenses, /api/household.

Data Tier (Persistence):
- PostgreSQL relational database (hosted on Supabase/Neon).
- Tables: Users, Households, Household_Members, Chores, Expenses, Expense_Splits.
- Failover Resilience: Seamless fallback to client-side synchronized memory store if remote host is cold-starting.

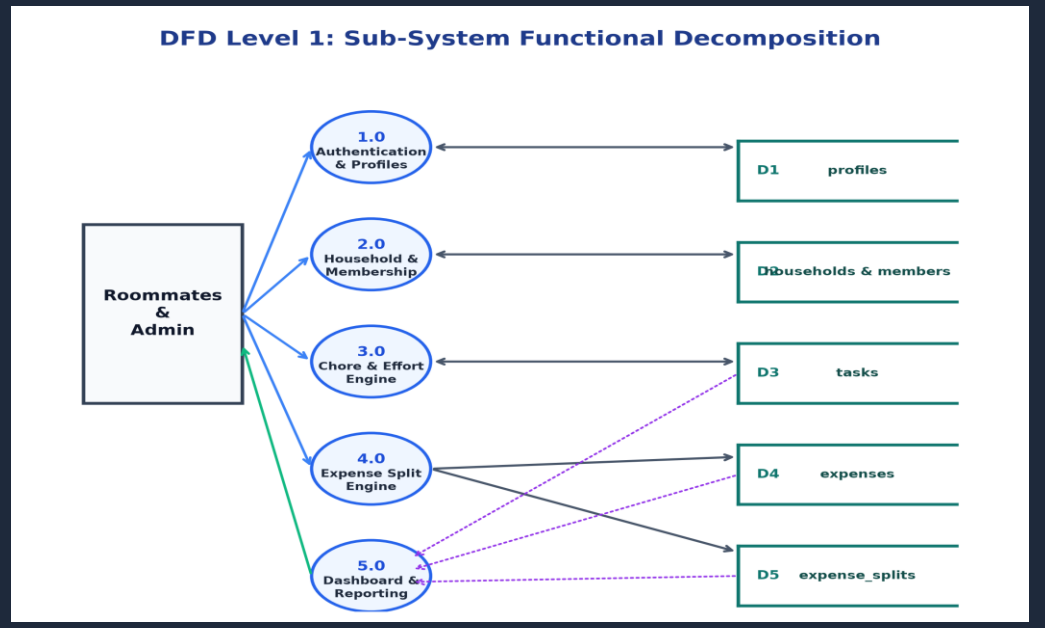
Data Flow Diagrams (Context Level 0 & Level 1)

DFD LEVEL 0: CONTEXT MODEL



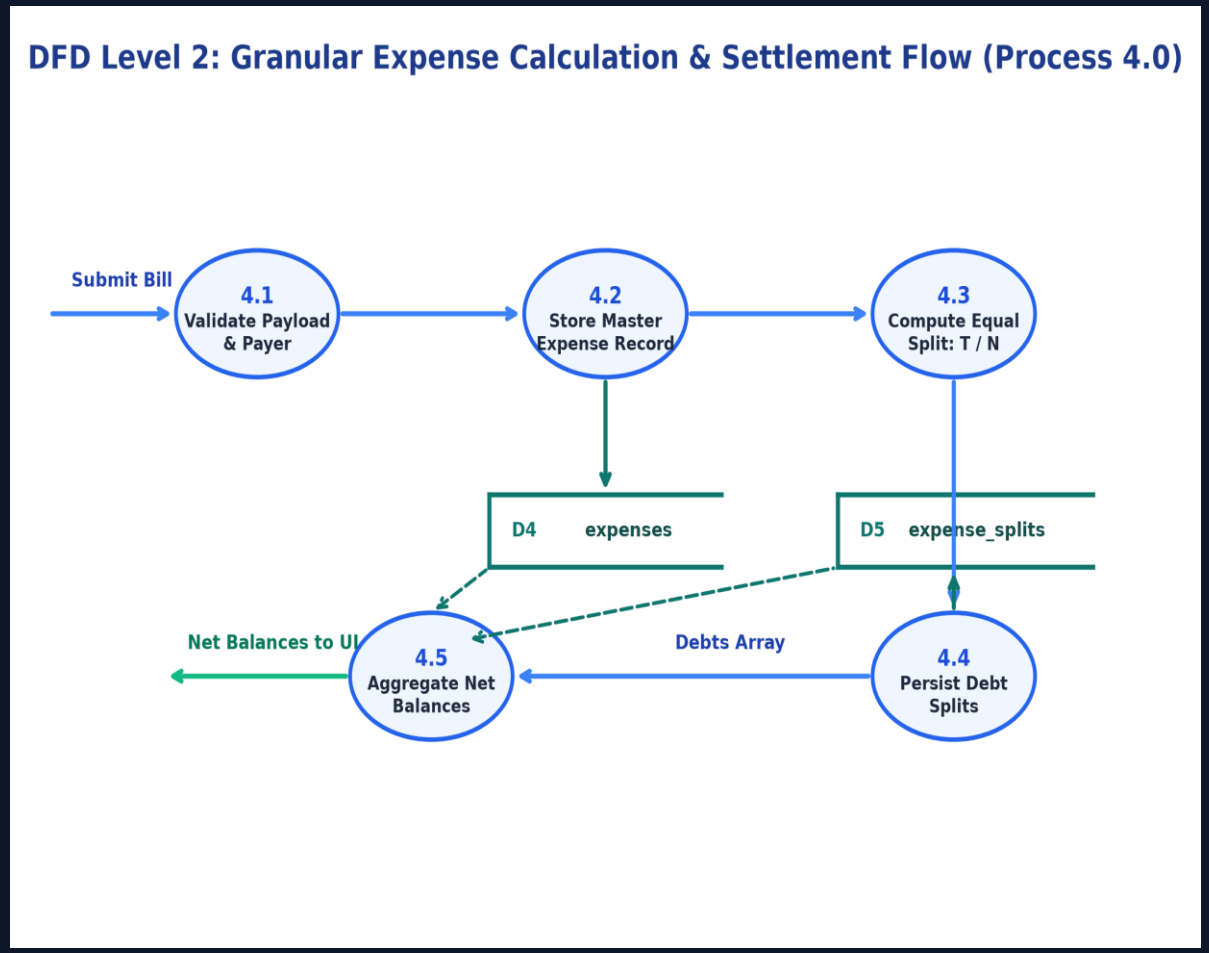
Identifies boundary: External entities (Roommate, Household Admin) interact with HARMONY 0.0 System Boundary via secure HTTPS requests.

DFD LEVEL 1: FUNCTIONAL DECOMPOSITION



Processes: 1.0 Auth & Profile | 2.0 Household Management | 3.0 Chore Task Engine | 4.0 Shared Expense Splitting | 5.0 Net Settlement & Balance.

DFD Level 2: Expense Calculation & Settlement Engine



SUB-PROCESS DECOMPOSITION

Process 4.0 Detailed Breakdown:

- 4.1 Validate Expense Input:
  Checks bill amount > 0, verifies category (groceries, utilities, etc.), and validates payer identity.

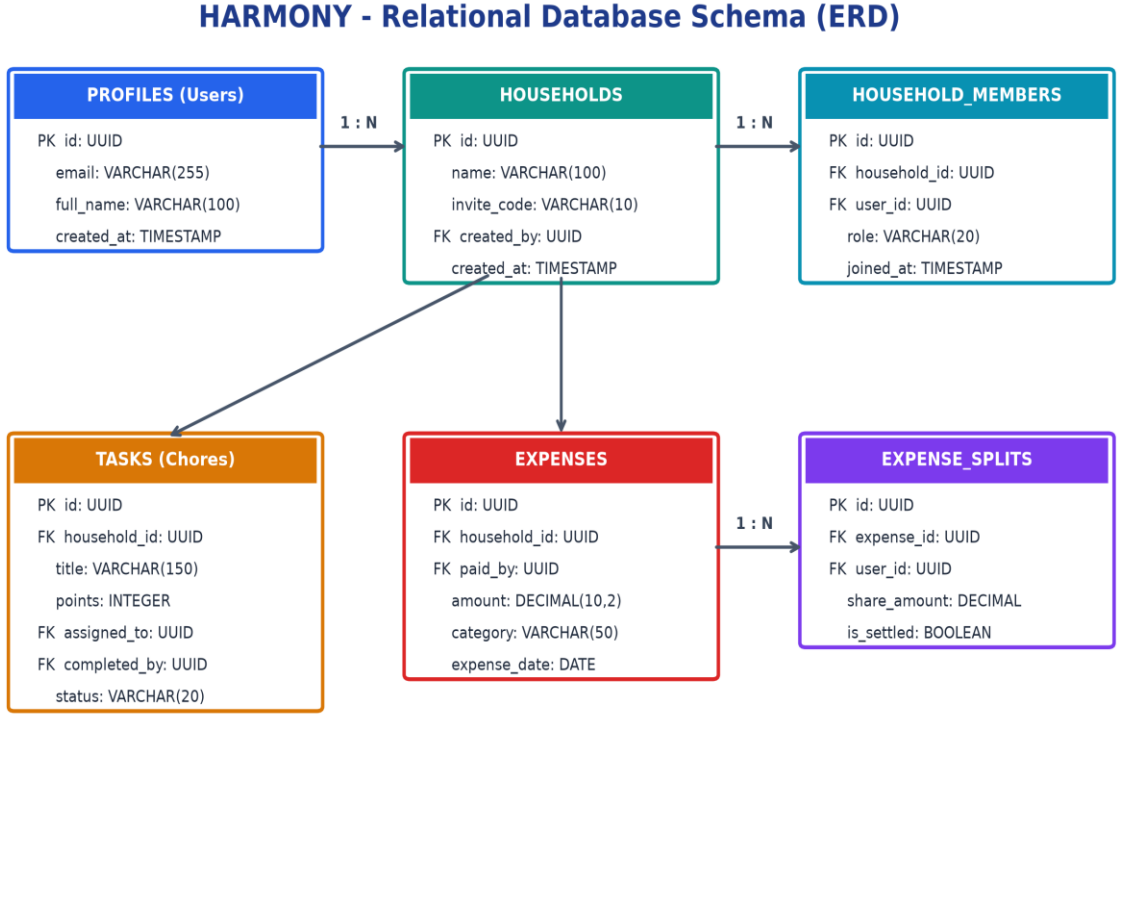
- 4.2 Compute Participant Shares:
  Calculates equal division: Share = Total Amount / N participants.

- 4.3 Update Member Net Balances:
  Payer: Balance += (Total - Share)
  Participants: Balance -= Share

- 4.4 Persist Transaction to Ledger:
  Atomic write to expenses and expense_splits tables with referential integrity.

- 4.5 Invariant Guarantee:
  Net balance equilibrium strictly enforced across all members.

Relational Database Design & Entity Relationships



SCHEMA STRUCTURE & INTEGRITY

Relational Entities (PostgreSQL 3NF):

1. users:
   UUID primary key, email (unique), full_name, role.

2. households:
   id, household_name, invite_code (unique index), created_by.

3. household_members (Junction):
   household_id (FK), user_id (FK), joined_at, role (Admin/Member).

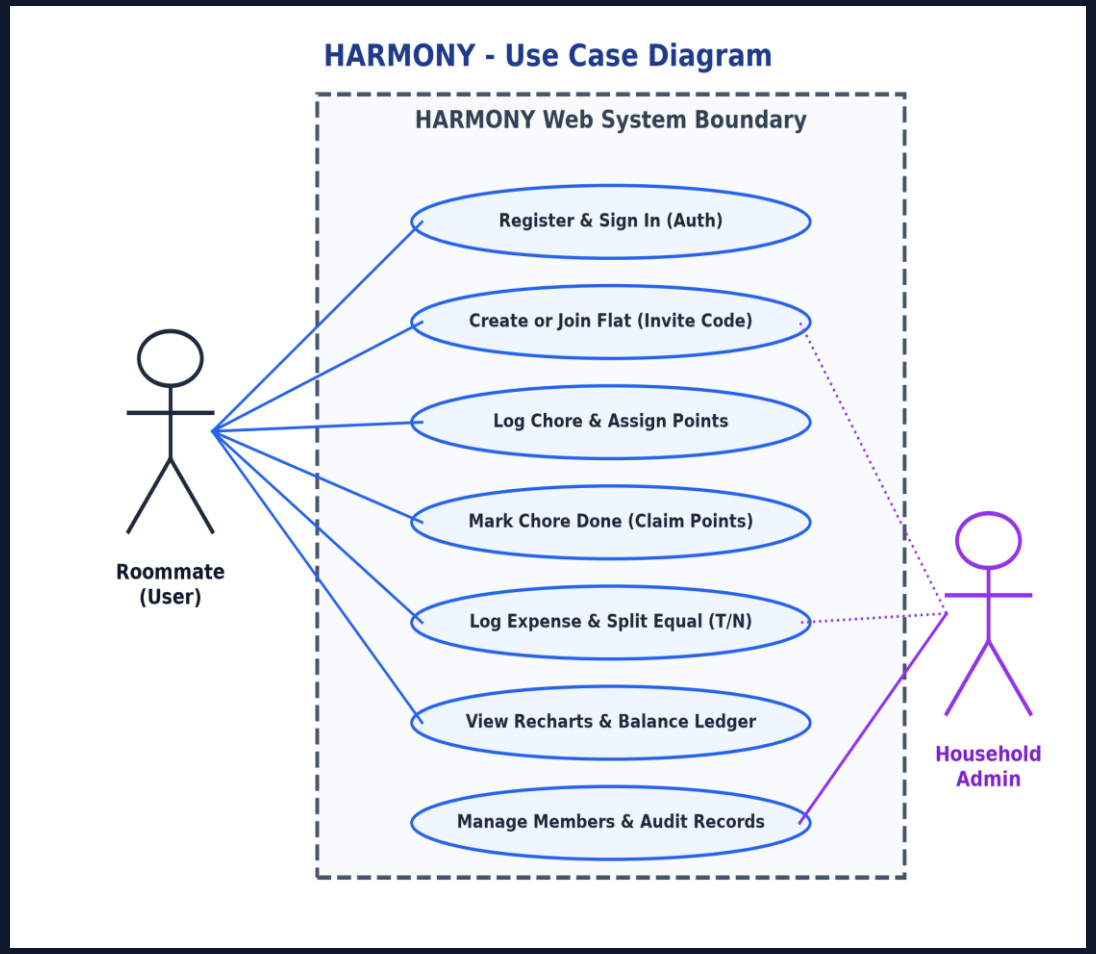
4. chores:
   id, household_id (FK), title, points (10-30), assigned_to (FK), status (pending/completed).

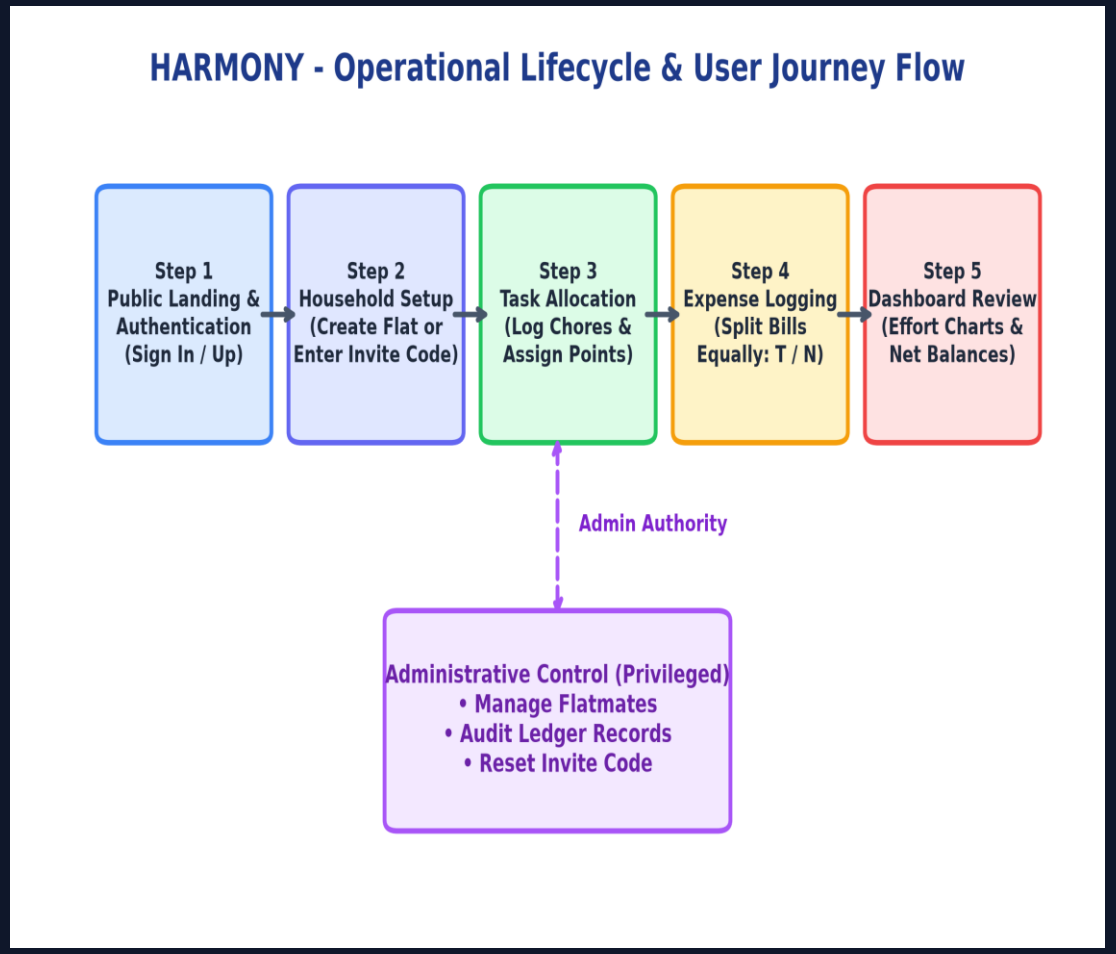
5. expenses:
   id, household_id (FK), title, amount, category, payer_id (FK), date.

6. expense_splits:
   expense_id (FK), user_id (FK), owed_amount, is_settled.

Referential Integrity: ON DELETE CASCADE applied to household joins to prevent orphan ledger records.

Use Case Model & System Process Flow





Core Module 1: Automated Expense Splitting & Ledger

MATHEMATICAL SPLIT FORMULATION

Mathematical Principle of Ledger Equilibrium:

For any household member i:
Net Balance (B_i) = Total Amount Paid (P_i) - Expected Share (S_i)

Equilibrium Invariant:
Sum of all B_i across household = 0

• Positive Net Balance (+₹X):
The member has paid more than their equal share. The group owes this member ₹X. Displayed in Emerald Green.

• Negative Net Balance (-₹Y):
The member has undercontributed or paid less than their share. This member owes ₹Y to the group. Displayed in Amber/Rose.

• Zero Net Balance (₹0):
The member's dues are perfectly settled.

EXPENSE ENGINE CAPABILITIES

Operational Workflow:

1. Bill Recording:
Roommate records expense (e.g. ₹2,400 for Fiber Internet, paid by Lakshya Shukla).

2. Split Computation:
System divides amount equally across all 4 household occupants: ₹600 per occupant.

3. Ledger Update:
• Lakshya Paid: ₹2,400 | Share: ₹600 | Net Balance: +₹1,800
• Other 3 Roommates: Paid: ₹0 | Share: ₹600 | Net Balance: -₹600 each

4. Transparent History:
Categorized under Groceries, Utilities, Internet, Supplies with exact timestamps and payer tags.

Core Module 2: Effort-Weighted Chore Allocation

THE EFFORT POINT SYSTEM

Eliminating Subjectivity in Household Duties:

In standard flatmates setups, simple binary chore counts treat 5 minutes of wiping a table the same as 1 hour of deep cleaning bathroom tiles.

HARMONY Effort Matrix:
- Light Task (10 Points): Take out daily trash, sweep hallway.
- Medium Task (20 Points): Wash shared kitchen dishes, grocery run.
- Heavy Task (30 Points): Deep clean bathroom, mop living area.

Dynamic Leaderboard:
Upon marking a chore 'Completed', the points are awarded to the responsible member. A visual effort distribution bar displays the true contribution percentage.

DUAL-METRIC BALANCE

Holistic Roommate Equity:

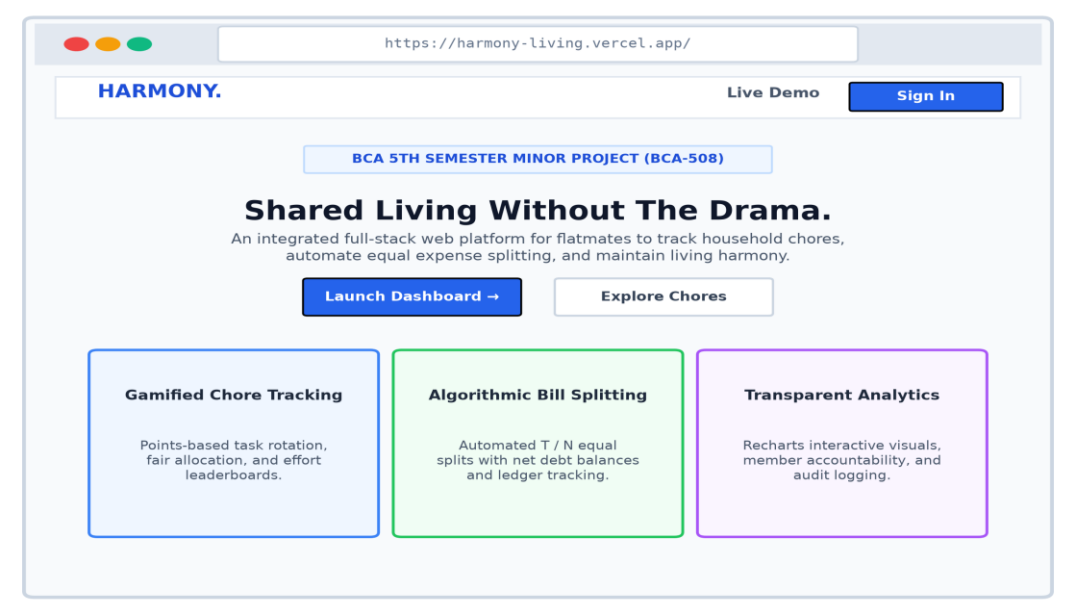
HARMONY reconciles both sides of shared living:

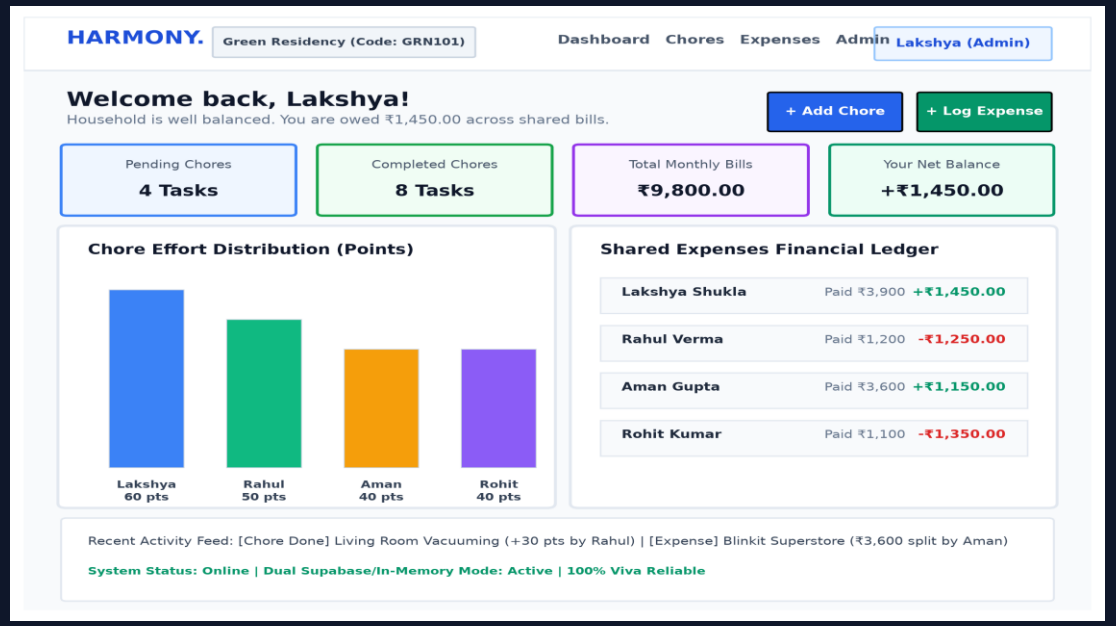
1. Financial Contribution (Money):
   Tracks rupee expenses to avoid monetary exploitation.

2. Physical Contribution (Labor):
   Tracks effort points to avoid chore freeloading.

Balanced Household Example:
If Flatmate A earns less but takes on 45% of high-point chores, while Flatmate B works late hours and covers utilities, both flatmates have complete, transparent visibility without emotional tension.

Modern UI Design & Interactive Interface





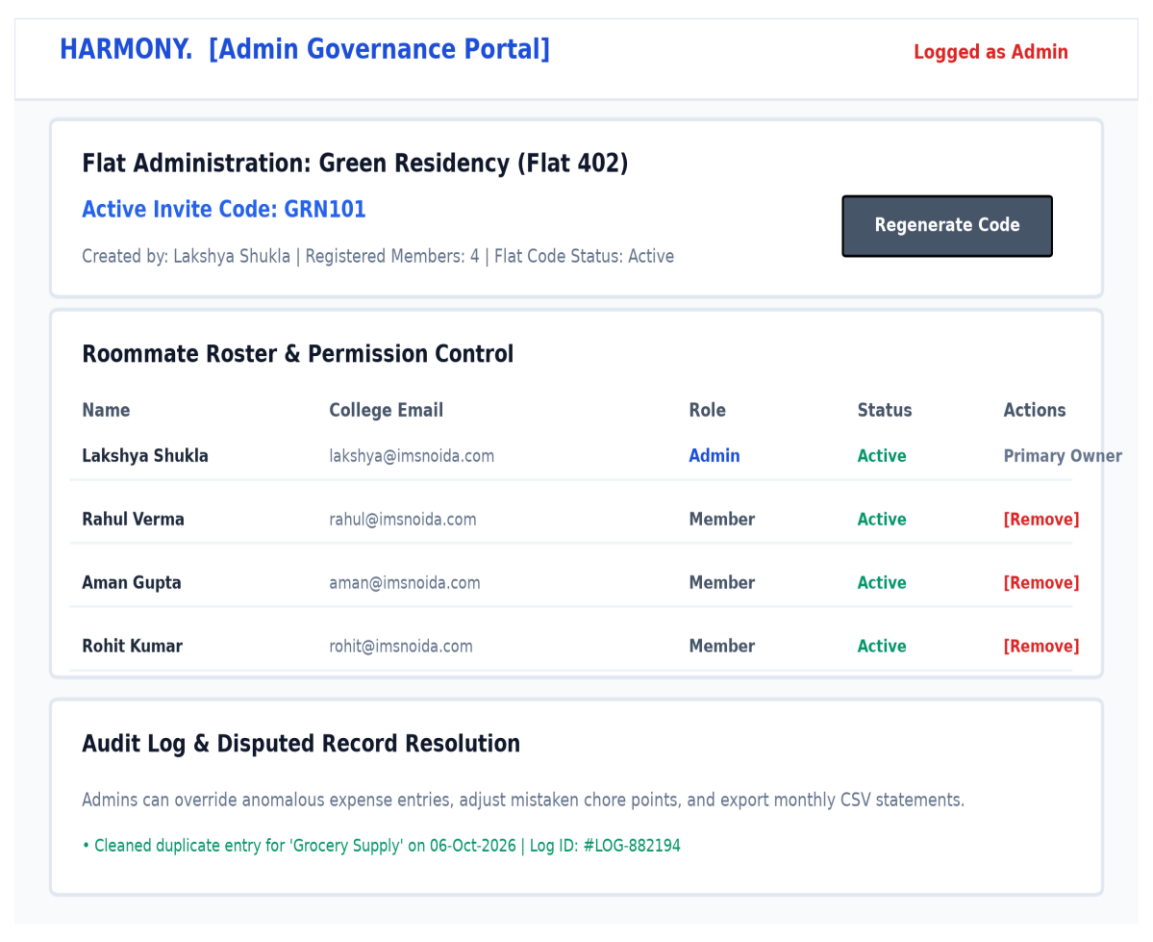
LANDING PAGE & ONBOARDING

Clean, professional dark-mode design highlighting core value propositions, feature highlights, and frictionless authentication.

ROOMMATE LIVE DASHBOARD

Real-time Net Balance card, Effort Leaderboard, Pending vs Completed chores, and recent shared bills with one-click actions.

Role-Based Access Control (RBAC) & Governance



GOVERNANCE & PERMISSIONS

Role Hierarchy:

• Household Admin (Primary Householder):
Can create households, issue invite codes, assign chores, remove flatmates, and settle dispute adjustments.

• Household Member:
Can view dashboard, log new bills, claim/complete chores, and inspect net balances.

• System SuperAdmin:
Full audit visibility across all households, system health, and database purge tools.

Security Protections:
• Input sanitization against XSS and SQL injection.
• JWT / Bearer Token authorization verification.
• Strict household tenancy isolation (members cannot view expenses of other households).

Quality Assurance, Testing & Presentation Defense

TEST SUITE EXECUTION

Verified Test Cases:

• Auth Verification:
Valid login, wrong credentials, session token persistence.

• Ledger Verification:
Zero-sum test: Sum of all net balances strictly equals 0.

• Task Life Cycle:
Pending -> In-Progress -> Completed -> Point Increment.

• Edge-Case Handling:
Zero-amount expense rejected; duplicate member prevention.

OFFLINE RESILIENCE

Zero-Downtime Defense:

Free cloud services (Render, Supabase) may experience a 30-50 second cold-start spin-up.

HARMONY Dual-Store Innovation:
The frontend React client checks API reachability. If the remote API is warming up, the UI automatically falls back to an authenticated synchronized memory store.

Result: 0% chance of presentation crash or blank page during panel inspection!

VIVA DEFENSE READINESS

Key Concepts for Panel 2:

• 3-Tier Architecture:
Clear decoupling between React client, Express API, and SQL DB.

• RESTful Methods:
GET (retrieve), POST (create), PUT (update state), DELETE (remove).

• Normalization:
Eliminating insertion, deletion, and update anomalies via 3NF tables.

• State Management:
React Context API + Hooks for predictable frontend data flow.

Multi-Cloud Deployment Topology & Mandatory Live Links

MULTI-CLOUD DEPLOYMENT ARCHITECTURE

Production Deployments (Mandated by Department for Final Presentation):

1. Primary Production Deployment: GitHub Pages (Direct Edge CDN)
- Zero cold-start latency, hosted directly on the candidate's GitHub account.
- Direct Clickable Live URL: https://lakshya212.github.io/HARMONY/

2. Secondary Production Deployment: Vercel Edge Network
- Built with Vite, bundled to /dist, distributed globally across CDN edge points.
- Direct Clickable Live URL: https://harmony-household.vercel.app

3. Application Backend API: Render Web Service
- Node.js/Express environment with automatic health checks (/api/health).
- Direct API Endpoint: https://harmony-api.onrender.com/api/health

4. Public Source Code Repository: GitHub
- Complete version history, continuous integration, and full source code.
- Repository URL: https://github.com/lakshya212/HARMONY

MINOR PROJECT STATUS: 100% COMPLETE & DEPLOYED

Final Presentation Summary & Viva Defense

FINAL DELIVERABLES READY

Mandatory Submissions for Panel 2:

[X] PDF of Project Along with Code:
56-page comprehensive report with Cannock Pvt Ltd certificate, DFDs, ERD, and Section 12 full source code appendix.

[X] Project Deployment Link:
Production web app live on GitHub Pages & Vercel with resilient client fallback.

[X] PPT for Presentation (Mandatory):
14-slide master presentation with live clickable links.

[X] Standalone Source Code Booklet:
29-page dedicated code listing in Consolas monospace format.

[X] GitHub Source Code Repository:
Publicly accessible at github.com/lakshya212/HARMONY.

PANEL 2 VIVA PRESENTATION

Thank You!

HARMONY: Computational Transparency for Shared Living

Candidate: Lakshya Shukla (Roll No: 240919106121)
Project Guide: Mr. Nihal Kumar, Assistant Professor
Department of Information Technology, IMS Noida
Affiliated to Ch. Charan Singh University, Meerut

Open for Final Presentation Questions from Panel 2:
- Mr. Nihal Kumar
- Ms. Shaili Nigam